

Encryption / Decryption Config and File Encryption Properties

As part of the [Arrow PME Extension Project](#), we extended the encryption and decryption configuration and file properties in order to pass new parameters along to the external service.

In this document, we specify the final additional parameters that we implemented for this purpose. With respect to the original proposal, this solution is offering:

- A new configuration where we can specify an encryption algorithm *per column* instead of an overall value for all.
- We are also leaving the door open to forward compatibility, where we can separate the concept of encryptor and encryption algorithm. For example, a user should be able to say they want to use an AES Encryptor, and specifically the GCM algorithm within.
- We simplified the amount of new classes needed, by collapsing the data previously contained in an external connection configuration into the extended encryption configuration. This simplifies the changes needed in the CryptoFactory class.
- We wrote a "[canonical Arrow application](#)" which intends to capture the preferred way to configure an application to use the new features. It can be used in combination with this document as a reference.
- **It is of particular importance to make this design as simple and intuitive as possible. We extended the Arrow classes for configuring encryption at a high level (Encryption / Decryption Config) and at a lower level used internally by Arrow (FileEncryption / FileDecryption Properties). If we are to integrate this code successfully with the main Arrow repository, we must respect the use of a clean and intuitive paradigm for specifying our desired parameters.**

Encryption

We extended the `EncryptionConfig` as:



A column is encrypted if the column name appears in either the `column_keys` or `per_column_encryption` map. The column name **should not** appear in both maps. The `CryptoFactory` method that constructs the `ExternalFileEncryptionProperties` performs this validation.

If the developer wants to use the external encryption agent, then **they must** use the `ExternalEncryptionConfig`.

To check whether a column should be encrypted, the `CryptoFactory` first checks the existing `column_keys` map. If the column name appears there, then:

1. Ask the KMS Connection service to resolve the `column_key_id`
2. Use the `encryption_algorithm` string to encrypt the column

If a column name appears in the new `per_column_encryption` map, then:

1. Ask the KMS Connection service to resolve the `encryption_key` (this is because the column's metadata will be encrypted using AES, so the KMS must resolve the key)
2. If the `encryption_algorithm` is "EXTERNAL", also forward the `encryption_key` to Arrow.
3. Use the `encryption_algorithm` and key specified within the `per_column_encryption` map.

The `app_context` dictionary contains metadata relevant to the external service. For some of the external services, like the Protegrity service, a user id **will be required**. Each external service implementation must validate the parameters it receives for correctness. This dictionary is purely meant to hold contextual information about the running application, like user ids, location, and security contexts.

The `configuration_properties` dictionary allows users to specify how to connect to, instantiate, and configure any type of external encryptor they wish to use. The map may contain an entry per each different type of encryptor. This dictionary is also the designated place for application developers to specify any additional parameters related to the encryptor itself, for example forcing per-block encryption, or specifying configuration file paths.

In that case the `configuration_properties` for DBPA might look like:

```
configuration_properties: {  
    "EXTERNAL_DBPA" : { "config_file" : "path/to/config/file", "force_block"  
: TRUE },  
}
```

Note that the current implementation of the DBPA server has full access to the `configuration_properties` as they are sent from the application. However, we only check the contents of this dictionary for the current DBPA uses, which are agent initialization and DLL loading. The data is accessible for reading on the DBPS side. However, if the DBPSEncryptor needs a specific flag or value, each implementer must add code to extract this value from the dictionary and forward it to the encryption logic. This means that while the dictionary values are present at the DBPS level, code is still required to parse and utilize these values for encryption.

The `CryptoFactory` has a new method that receives an `ExternalEncryptionConfig` as a parameter, and constructs an `ExternalFileEncryptionProperties`. Likewise, we extend the original `FileEncryptionProperties` as:



We extended the [ColumnEncryptionProperties](#) class to specify which encryption algorithm to use per each column. This way we can reuse the `encrypted_columns` map in the original class and add the `per_column_encryption` configuration we received in the `ExternalEncryptionConfig` to the already existing `encrypted_columns` map. Therefore, we only need to add the `app_context` and the `configuration_properties` to the new subclass.

Decryption

When it comes to decryption, the extra information we send to the external service is the same additional information that we sent for the encryption case. So we extend the `DecryptionConfig` as:



Note that we do not send the per column encryption information, in keeping with the standard practices currently in place for decryption. This means that we read each column's encryption information from the Parquet file metadata instead.

The `CryptoFactory` adds a new method that receives an `ExternalDecryptionConfig` as a parameter, and constructs an `ExternalFileDecryptionProperties`. Likewise, we extend the original `FileDecryptionProperties` as:



As in the case with the encryption, we extend [ColumnDecryptionProperties](#) to include the encryption algorithm used.

Potential Future Improvements - Metadata Encryption

Note that we mentioned that any column appearing in the `per_column_encryption` map must be able to be resolved by the KMS system. KMS resolution remains mandatory for all encrypted columns because the column metadata is currently always encrypted using the default AES implementation, which relies on the KMS to resolve the necessary keys from the key metadata stored in the file.

During decryption, while the external service might successfully decrypt the column data, the subsequent decryption of the column metadata fails. This is because we are keeping the default Arrow behavior where the column metadata decryption defaults to the standard `AesDecryptor`, which requires the full JSON key metadata structure to perform a lookup and resolve the key via KMS.

The root cause of this problem is the fact that we have different types of encryptors/decryptors for the column data and for the column metadata. This is a current limitation of the `ExternalDBPA`, because it does not support encryption and decryption of metadata. This may change in the future if DBPA enables metadata encryption and decryption. Another potential solution would be to introduce a `per_column_metadata_encryption` map, which would require Arrow modifications to check in each

separate map for the correct encryptor and decryptor depending on whether we are working on data or metadata.

Appendix: Diagram Code

Used in planttext.com

```
@startuml
class EncryptionConfig {
    - footer_key: string
    - column_keys: &#123;
        "column_key_id_1": ["column_name_1", ..., "column_name_n"],
        ...,
        "column_key_id_n": ["column_name_2"]
    &#125;
    - encryption_algorithm: string
    - cache_lifetime: datetime
    - data_key_length_bits: int
    - plaintext_footer: bool
}

class ExternalEncryptionConfig {
    - per_column_encryption: &#123;
        "column_name_3" : &#123; "encryption_algorithm": "AES_GCM", "encryption_key":
"key_1" &#125;;
        ...,
        "column_name_4" : &#123; "encryption_algorithm": "EXTERNAL_DBPA",
"encryption_key": "key_n" &#125;
    &#125;

    - app_context: &#123;
        "user_id" : "Picard1701", ** // required in some cases**
        ...,
        "location" : "Presidio"
    &#125;

    - configuration_properties: &#123;
        "EXTERNAL_DBPA" : &#123; "config_file": "path/to/config/file" &#125;
        "EXTERNAL_OTHER_TYPE" : &#123; "config_file": "path/to/config/file",
"config_file_decryption_key": "some_key" &#125;
    &#125;
}
"EncryptionConfig" <|-- ExternalEncryptionConfig
@enduml

@startuml
class FileEncryptionProperties {
    - encryption_algorithm: enum
    - encrypted_footer: bool
    - footer_key: string
    - footer_key_metadata: string
}
```

```

- aad_prefix: string
- store_aad_prefix_in_file: bool
- encrypted_columns: &#123;
    "column_name_1": shared_ptr<ColumnEncryptionProperties>,
    ...,
    "column_name_n": shared_ptr<ColumnEncryptionProperties>
&#125;
}

class ExternalFileEncryptionProperties {
- app_context: &#123;
    "user_id" : "Picard1701",
    "schema" : "some_value",
    ...,
    "location" : "Presidio"
&#125;

- configuration_properties: &#123;
    "EXTERNAL_DBPA" : &#123; "config_file": "path/to/config/file" &#125;
    "EXTERNAL_OTHER_TYPE" : &#123; "config_file": "path/to/config/file",
"config_file_decryption_key": "some_key" &#125;
    &#125;
}
"FileEncryptionProperties" <|-- ExternalFileEncryptionProperties
@enduml

@startuml
class DecryptionConfig {
- cache_lifetime_seconds: double
}
class ExternalDecryptionConfig {
- app_context: &#123;
    "user_id" : "Picard1701", ** // required in some cases**
    ...,
    "location" : "Presidio"
&#125;

- configuration_properties: &#123;
    "EXTERNAL_DBPA" : &#123; "config_file": "path/to/config/file" &#125;
    "EXTERNAL_OTHER_TYPE" : &#123; "config_file": "path/to/config/file",
"config_file_decryption_key": "some_key" &#125;
    &#125;
}
"DecryptionConfig" <|-- ExternalDecryptionConfig
@enduml

@startuml
class FileDecryptionProperties {
- footer_key: SecureString
- aad_prefix: string
- aad_prefix_verifier: AADPrefixVerifier
- key_retriever: DecryptionKeyRetriever

```



```

- check_plaintext_footer_integrity: bool
- plaintext_files_allowed: bool
- column_decryption_properties: &#123;
    "column_name_1": shared_ptr<ColumnDecryptionProperties>,
    ...,
    "column_name_n": shared_ptr<ColumnDecryptionProperties>
&#125;
}
class ExternalFileDecryptionProperties {
- app_context: &#123;
    "user_id" : "Picard1701",
    "schema" : "some_value",
    ...,
    "location" : "Presidio"
&#125;

- configuration_properties: &#123;
    "EXTERNAL_DBPA" : &#123; "config_file": "path/to/config/file" &#125;
    "EXTERNAL_OTHER_TYPE" : &#123; "config_file": "path/to/config/file",
"config_file_decryption_key": "some_key" &#125;
&#125;

}
"FileDecryptionProperties" <|-- ExternalFileDecryptionProperties
@enduml

```

<>